# Manual de Instalación - AlertaMujer

**Versión:** 2.1
**Fecha:** 28 de octubre de 2026
**Audiencia:** Administradores de sistemas, DevOps y personal de TI
**Entorno:** Desarrollo local y producción

---

## Tabla de Contenidos


1. Requisitos del Sistema
2. Instalación del Backend en Local
3. Instalación del Frontend Web
4. Instalación de la App Móvil
5. Configuración de la Base de Datos
6. Despliegue en Producción (Render, Vercel y EAS)
7. Despliegue en Servidor Propio
8. Troubleshooting
9. Mantenimiento
10. Análisis Multiplataforma
11. Apéndice


---

## 1. Requisitos del Sistema

AlertaMujer tiene tres partes que se instalan por separado:

- **Backend:** tres microservicios Spring Boot 3.2.0 (Java 17): **Auth**, **Alert** y **Evidence**, sobre una base PostgreSQL 16. Repositorio Backend-ALERTA-MUJER, carpeta microservicios/.
- **Frontend web:** Angular 21. Repositorio Frond-end-Web, carpeta alerta-mujer-web.
- **App móvil:** Expo SDK 54 / React Native 0.81. Repositorio AlertaMujer, carpeta Frontend-movil.

El monolito anterior (backend-monolith/), el gateway y la carpeta db/ se eliminaron del repositorio el 25/10/2026. Producción se publica desde la rama main de los tres repositorios.

Hoy el proyecto está desplegado así: los tres microservicios y la base en **Render**, las evidencias en **Backblaze B2**, la web en **Vercel** y el APK generado con **EAS Build** (o compilado en local). Este manual explica ese despliegue (sección 6) y también cómo instalar todo en un servidor propio con Docker y Nginx (sección 7).

### 1.1 Requisitos de Hardware

**Servidor para los 3 microservicios + PostgreSQL (mínimo):** - CPU: 2 núcleos - RAM: 2 GB (cada servicio es una JVM aparte) - Disco: 20 GB SSD - Red: 100 Mbps

**Servidor recomendado:** - CPU: 4 núcleos - RAM: 4 GB - Disco: 50 GB SSD (más el almacenamiento de evidencias si no se usa Backblaze B2)

**Frontend web:** sitio estático (HTML, CSS y JS) de menos de 100 MB. Puede ir en el mismo servidor detrás de Nginx o en Vercel.

**Equipo para compilar o desarrollar:** - RAM: 8 GB recomendados (Maven, Angular y Gradle consumen bastante al compilar) - Disco: 15 GB libres para dependencias (node_modules, repositorio de Maven, imágenes de Docker, Android SDK)

**Celular de la usuaria:** - Android 7.0 (API 24) o superior (probado en un Samsung SM-A125M y un Infinix X678B) - GPS, cámara y micrófono

### 1.2 Requisitos de Software

| Herramienta | Versión | Para qué |
|---|---|---|
| Node.js + npm | 20 o superior | Web y móvil |
| Java JDK | 17 | Microservicios sin Docker y APK en local |
| Maven | 3.9 | Microservicios sin Docker |
| Docker + Docker Compose | 20.10+ / 2.0+ | PostgreSQL, SeaweedFS (S3 local) y los 3 servicios |
| Git | Reciente | Clonar los repositorios |
| Android SDK | Reciente | APK en local (eas build --local) |
| EAS CLI + cuenta de Expo | 24.12 o superior | APK en la nube |
| Nginx | 1.18+ | Solo si se usa servidor propio |

**Sistemas operativos:** Ubuntu 22.04 LTS o superior (recomendado para servidor), Debian 11+, Windows 10/11 con Docker Desktop y WSL2, o macOS 11+.

### 1.3 Cuentas y servicios externos

| Servicio | Para qué |
|---|---|
| Firebase | Inicio de sesión con Google y notificaciones push (FCM). Se necesita el archivo de cuenta de servicio para Auth y Alert, y google-services.json para la app |
| Brevo o Resend | Envío de correos por API HTTP (aviso a contactos, fin de alerta y código de recuperación) |
| Backblaze B2 | Almacenamiento de evidencias compatible |

| Servicio | Para qué |
|---|---|
| | con S3 |
| Render | Microservicios y base de datos en producción |
| Vercel | Web en producción |
| Expo (EAS) | Compilación del APK en la nube |

Ya no se necesita clave de Google Maps: el móvil usa MapLibre y la web usa Leaflet con OpenStreetMap.

### 1.4 Requisitos de Red

| Servicio | Puerto | Descripción |
|---|---|---|
| Auth Service | 8081 | API REST (local) |
| Alert Service | 8082 | API REST (local) |
| Evidence Service | 8083 | API REST (local) |
| PostgreSQL | 5432 | Base de datos dentro de Docker (interno) |
| Angular (desarrollo) | 4200 | npm start |
| Expo / Metro (desarrollo) | 8081 por defecto | Choca con Auth si corren en el mismo equipo: usa otro puerto (ver 4.2) |
| HTTP / HTTPS | 80 / 443 | Solo servidor propio |

**Salida a internet necesaria:** Firebase, el proveedor de correo, Backblaze B2 y los servidores de mapas de OpenStreetMap.

### 1.5 Credenciales y Secretos

- JWT_SECRET largo y aleatorio. Los servicios **no arrancan** si está vacío o si es el valor que estuvo publicado en el repositorio.
- INTERNAL_API_KEY larga y aleatoria, igual en los tres servicios.
- Contraseñas fuertes para PostgreSQL.
- Archivo de cuenta de servicio de Firebase. Nunca se sube al repositorio.
- Clave del proveedor de correo y claves de Backblaze B2.

Ningún valor real se escribe en este manual. Pídelos al equipo responsable.

Generar valores aleatorios:

```
openssl rand -base64 64   # JWT_SECRET
openssl rand -hex 32      # INTERNAL_API_KEY
openssl rand -hex 24      # contraseñas de base de datos
```

## 2. Instalación del Backend en Local

### 2.1 Preparación (Ubuntu)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg git

# Repositorio de Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker

# Opcional: usar Docker sin sudo
sudo usermod -aG docker $USER
newgrp docker

docker --version
docker compose version
```

En Windows y macOS instala Docker Desktop (ver sección 10).

### 2.2 Obtener el Código

```
git clone <url-repositorio-backend> Backend-ALERTA-MUJER
cd Backend-ALERTA-MUJER
git checkout <rama-indicada-por-el-equipo>
```

Producción se publica desde la rama main. Para desarrollar, crea tu rama desde develop: los cambios entran por pull request a develop y se promueven a qa y a main.

### 2.3 Configurar Variables de Entorno

Crea el archivo .env a partir de .env.example y llena los valores:

```
cp .env.example .env
nano .env
```

Variables que usan los microservicios (solo nombres):

| Variable | Servicios | Descripción |
|---|---|---|
| JWT_SECRET | Todos | Clave de firma del JWT. La |

| Variable | Servicios | Descripción |
|---|---|---|
| | | misma en los tres |
| INTERNAL_API_KEY | Todos | Clave de la API interna (X-Internal-Key). La misma en los tres |
| CORS_ORIGIN | Todos | Origen permitido de la web (en local, http://localhost:4200) |
| DB_HOST, DB_PORT, DB_NAME y credenciales | Todos | Conexión a PostgreSQL |
| SERVICES_AUTH_URL, SERVICES_ALERT_URL, SERVICES_EVIDENCE_URL | Todos | URL con las que los servicios se llaman entre sí |
| Archivo de Firebase | Auth y Alert | Cuenta de servicio para Google y push |
| EMAIL_PROVIDER, EMAIL_API_KEY, EMAIL_FROM | Alert | Proveedor de correo (Resend en uso; Brevo como alterno), clave y remitente |
| notifications.web-base-url | Alert | URL de la web, para armar el enlace de seguimiento |
| Endpoint, bucket y claves S3 | Evidence | Almacenamiento de evidencias (SeaweedFS en local, Backblaze B2 en producción) |

POSTGRES_PASSWORD, APP_DB_PASSWORD, JWT_SECRET e INTERNAL_API_KEY no tienen valor por defecto: Docker Compose se detiene con un mensaje si falta alguno, y los servicios no arrancan sin ellos.

En desarrollo puedes dejar sin configurar el proveedor de correo: los correos se escriben en el log del servicio Alert y no se envían.

### 2.4 Agregar el Archivo de Firebase

Pide al equipo el archivo de cuenta de servicio (se descarga en Firebase Console > Configuración del proyecto > Cuentas de servicio). Guárdalo fuera del código, en la ruta que espera docker-compose.yml para Auth y Alert, y verifica que esa ruta esté en .gitignore.

```
chmod 600 <ruta-del-archivo-de-firebase>
```

### 2.5 Levantar los Servicios con Docker

```
docker compose up --build
```

El archivo docker-compose.yml levanta:

- **PostgreSQL 16** (puerto 5433 del equipo)
- **SeaweedFS**, un almacenamiento compatible con S3 que reemplaza a Backblaze B2 en local
- **Auth Service** en el puerto 8081
- **Alert Service** en el puerto 8082
- **Evidence Service** en el puerto 8083

En el primer arranque, **Auth** aplica las migraciones de Liquibase: crea los esquemas identity, alert, resource, admin y notification, las 35 tablas, los índices, los triggers de auditoría y el usuario de la aplicación. Alert y Evidence no tocan el esquema.

### 2.6 Verificar

```
docker compose ps

curl http://localhost:8081/actuator/health
curl http://localhost:8082/actuator/health
curl http://localhost:8083/actuator/health
```

Cada uno debe responder:

```
{"status":"UP"}
```

Comprobaciones de seguridad:

```
# Sin token debe responder 403
curl -i http://localhost:8081/api/users/me

# La API interna sin clave debe responder 403
curl -i -X POST http://localhost:8081/internal/rpc/IdentityApi/prueba
```

### 2.7 Ejecutar las Pruebas

```
cd microservicios
mvn clean test
```

Resultado esperado: 351 pruebas que pasan (shared-kernel 16, Auth 114, Alert 196, Evidence 25), incluidas las pruebas de arquitectura con ArchUnit. GitHub Actions las ejecuta en cada push y pull request.

### 2.8 Backend sin Docker

1. Instala Java 17, Maven 3.9 y PostgreSQL 16 (sección 5.2).
2. Crea la base de datos.
3. Exporta las variables de la sección 2.3.
4. Compila y arranca cada servicio, empezando por Auth (aplica las migraciones):

```
cd microservicios
mvn clean package -DskipTests
```

```
java -jar auth-service/target/*.jar
java -jar alert-service/target/*.jar
java -jar evidence-service/target/*.jar
```

Cada servicio se ejecuta en su propia terminal o como servicio de systemd. Para las evidencias necesitas un almacenamiento S3 (SeaweedFS, MinIO o Backblaze B2).

### 2.9 Crear el Primer Administrador

En una base de producción no hay cuentas. Para tener un administrador:

1. Regístrate normalmente desde la app o la web.
2. Entra a la base y cambia el rol de esa cuenta:

```
UPDATE identity.users SET role_id = 2 WHERE email = 'correo-del-admin@ejemplo.com';
```

3. Cierra sesión y vuelve a entrar para que el token lleve el rol nuevo.

---

## 3. Instalación del Frontend Web

### 3.1 Instalar Node.js

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node --version
npm --version
```

### 3.2 Obtener e Instalar

```
git clone <url-repositorio-web> Frond-end-Web
cd Frond-end-Web/alerta-mujer-web
npm ci
```

Instala **sin** --production: Angular necesita las dependencias de desarrollo para compilar.

### 3.3 Configurar las URL de los Servicios

La web no lee un .env. Las URL van en src/environments/environment.ts (desarrollo) y src/environments/environment.prod.ts (producción):

```
export const environment = {
  production: true,
  apiUrl: 'https://<url-de-auth>',
  alertApiUrl: 'https://<url-de-alert>',
  evidenceApiUrl: 'https://<url-de-evidence>',
  firebase: {
    apiKey: '',
    authDomain: '',
    projectId: '',
```

```
  appId: '',
 },
};
```

- Las URL llevan solo el dominio, sin /api al final.
- El interceptor elige el servicio según el inicio de la ruta (/api/evidences va a Evidence; alertas, contactos, ubicaciones, zonas y recursos van a Alert; el resto a Auth).
- El bloque firebase se llena con los datos de la app web en Firebase Console. Además, el dominio de la web debe estar en Firebase Console > Authentication > Settings > Authorized domains.

### 3.4 Desarrollo, Pruebas y Build

```
npm start                    # http://localhost:4200
npx ng test --watch=false    # 25 pruebas
npm run build                # salida en dist/alerta-mujer-web/browser
```

---

## 4. Instalación de la App Móvil

La app no se instala en el servidor: se compila un APK y se instala en el celular.

### 4.1 Obtener e Instalar Dependencias

```
git clone <url-repositorio-movil> AlertaMujer
cd AlertaMujer/Frontend-movil
npm install
```

Coloca google-services.json (lo entrega el equipo, desde Firebase Console) en la raíz de Frontend-movil. Lo necesitan las notificaciones push y el inicio de sesión con Google.

### 4.2 Desarrollo

```
npm start
```

npm start ejecuta scripts/update-env.js, que detecta la IP del computador y escribe en .env:

```
EXPO_PUBLIC_API_URL=http://192.168.1.X:8081
EXPO_PUBLIC_ALERT_API_URL=http://192.168.1.X:8082
EXPO_PUBLIC_EVIDENCE_API_URL=http://192.168.1.X:8083
```

- El celular y el computador deben estar en la misma red Wi-Fi.
- Para el emulador de Android se usa 10.0.2.2 en lugar de la IP.
- Metro usa por defecto el puerto 8081, el mismo de Auth. Si Expo avisa que el puerto está ocupado, acepta el puerto que te ofrece.
- La app usa módulos nativos, así que no funciona en Expo Go. Usa un APK de desarrollo (perfil development).

Pruebas:

```
npm test            # 73 pruebas
npx tsc --noEmit    # tipos
```

### 4.3 Perfiles de EAS

En eas.json, los perfiles preview y production ya traen las URL de los tres servicios de Render:

```
"preview": {
  "distribution": "internal",
  "android": { "buildType": "apk" },
  "env": {
    "EXPO_PUBLIC_API_URL": "https://alerta-mujer-auth.onrender.com",
    "EXPO_PUBLIC_ALERT_API_URL": "https://alerta-mujer-alert.onrender.com",
    "EXPO_PUBLIC_EVIDENCE_API_URL": "https://alerta-mujer-evidence.onrender.com"
  }
}
```

En estos perfiles el tráfico HTTP sin cifrar está desactivado: las URL siempre deben empezar por https://.

### 4.4 Generar el APK

**En la nube (EAS):**

```
npm install -g eas-cli
eas login
eas build -p android --profile preview
```

Al terminar, EAS entrega un enlace para descargar el APK. La cuota gratuita de EAS se agotó hasta el 1 de noviembre de 2026; mientras tanto se compila en local.

**En local** (necesita Android SDK y JDK 17):

```
npx eas-cli build -p android --profile preview --local
```

El APK queda en la carpeta del proyecto.

**Para Play Store** se usa el perfil production, que genera un .aab. Este build todavía no se ha probado:

```
eas build -p android --profile production
```

### 4.5 Instalar en el Celular

1. Copia o descarga el APK en el celular (o usa adb install <archivo>.apk).
2. Si Android avisa que no puede instalar apps de fuentes desconocidas, permítelo para el navegador o el gestor de archivos.
3. Instala y abre AlertaMujer.
4. En la app, entra a **Permisos y protección** y activa lo que falte.

**APK publicado:** la versión 1.0.1 está en GitHub Releases (28/10/2026, con los tiempos de 15, 25 y 35 minutos de la protección automática), compilada desde main con las URL de los tres servicios de Render. El botón **Descargar APK** de la web apunta siempre al último release.

---

## 5. Configuración de la Base de Datos

### 5.1 Migraciones con Liquibase

La base **no** se crea con un script init.sql. Al arrancar, **Auth Service** aplica las migraciones de db/changelog/:

- **v1.0.0:** esquemas identity, alert, resource y admin con sus tablas
- **v1.1.0:** índices, restricciones, auditoría y usuario de la aplicación con permisos limitados
- **v1.2.0:** recorridos guardados
- **v1.3.0:** archivos de evidencias
- **v1.4.0:** roles y configuración base
- **Changesets posteriores:** aviso a contactos, seguimiento, chat, eliminación de cuentas, sesiones y retención

El contexto se elige con la configuración de Liquibase: - dev: carga además datos de prueba. Solo para desarrollo. - prod: solo roles y configuración base.

La carpeta db/ con init.sql se eliminó del repositorio: el esquema lo crea solo Liquibase. El script de estructura actualizado está en el Documento 06 (alerta_mujer_schema.sql).

### 5.2 Instalación Manual de PostgreSQL 16 (sin Docker)

```
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc --fail
https://www.postgresql.org/media/keys/ACCC4CF8.asc
echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc]
https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" | sudo tee
/etc/apt/sources.list.d/pgdg.list
sudo apt update
sudo apt install -y postgresql-16
sudo systemctl enable --now postgresql
```

Crear el usuario dueño y la base:

```
sudo -u postgres psql

CREATE USER alerta_admin WITH PASSWORD '********';
CREATE DATABASE alerta_mujer OWNER alerta_admin;
\q
```

El usuario dueño es el que usa Liquibase. El usuario con el que corren los servicios lo crea la migración.

### 5.3 Verificar las Migraciones

```
psql -h <host> -U alerta_admin -d alerta_mujer -c "\dn"
psql -h <host> -U alerta_admin -d alerta_mujer -c "SELECT id, filename, dateexecuted FROM databasechangelog ORDER BY orderexecuted DESC LIMIT 5;"
```

### 5.4 Backups

**Base de datos:**

```
#!/bin/bash
BACKUP_DIR="/var/backups/alertamujer"
DATE=$(date +%Y%m%d_%H%M%S)
mkdir -p "$BACKUP_DIR"

pg_dump "<cadena-de-conexión>" | gzip > "$BACKUP_DIR/alertamujer_$DATE.sql.gz"

# Mantener los últimos 7 días
find "$BACKUP_DIR" -name "*.gz" -mtime +7 -delete
```

Guárdalo como /usr/local/bin/backup-alertamujer.sh, dale permisos (chmod +x) y prográmalo con cron:

0 2 * * * /usr/local/bin/backup-alertamujer.sh

**Evidencias:** en producción están en Backblaze B2, que ya es persistente. Si se usa SeaweedFS o MinIO en un servidor propio, haz backup del volumen del almacenamiento.

**Base de Render:** la base gratuita no tiene backups automáticos y tiene fecha de vencimiento. Haz un pg_dump con la URL externa que da Render antes de esa fecha.

---

## 6. Despliegue en Producción (Render, Vercel y EAS)

Así está desplegado el proyecto hoy.

| Componente | Dirección |
|---|---|
| Web | https://alerta-mujer.vercel.app |
| Auth Service | https://alerta-mujer-auth.onrender.com |
| Alert Service | https://alerta-mujer-alert.onrender.com |
| Evidence Service | https://alerta-mujer-evidence.onrender.com |

### 6.1 Microservicios y Base de Datos en Render

El archivo render.yaml de la rama main describe los tres servicios Docker (Root Directory microservicios), la base PostgreSQL 16 y un grupo de variables compartidas, para que los secretos se configuren en un solo lugar. La producción actual se configuró desde el panel de Render con esos mismos valores.

**Paso 1:** En https://dashboard.render.com elige **New > Blueprint**, selecciona el repositorio Backend-ALERTA-MUJER y la rama main.

**Paso 2:** Pulsa **Apply**.

**Paso 3:** En el grupo de variables, configura: - JWT_SECRET e INTERNAL_API_KEY (valores largos y aleatorios) - CORS_ORIGIN con la URL de la web en Vercel - SERVICES_AUTH_URL, SERVICES_ALERT_URL y SERVICES_EVIDENCE_URL con las URL de los tres servicios - Los datos de conexión de la base

**Paso 4:** En **Alert**, configura el correo: EMAIL_PROVIDER (Resend en uso; Brevo alterno), EMAIL_API_KEY, EMAIL_FROM y la URL de la web para el enlace de seguimiento. Render bloquea SMTP, por eso el correo se envía por API HTTP.

**Paso 5:** En **Evidence**, configura el endpoint, el bucket y las claves de Backblaze B2.

**Paso 6:** En **Auth** y en **Alert**, entra a **Environment > Secret Files** y agrega el archivo de cuenta de servicio de Firebase.

**Paso 7:** Despliega **Auth** primero (aplica las migraciones) y luego Alert y Evidence: **Manual Deploy > Deploy latest commit**.

**Paso 8:** Verifica:

curl https://alerta-mujer-auth.onrender.com/actuator/health
curl https://alerta-mujer-alert.onrender.com/actuator/health
curl https://alerta-mujer-evidence.onrender.com/actuator/health

**Paso 9:** Crea el primer administrador (sección 2.9). La conexión a la base se copia desde la base de datos en Render > **Connect > PSQL Command**.

**Paso 10:** Carga zonas y recursos de ayuda desde el panel de administración (**Zonas y puntos de ayuda**). En una base nueva están vacíos y el mapa sale sin datos.

**Paso 11:** Haz una alerta de prueba real y confirma que llegan el correo y la notificación push al contacto.

Límites del plan gratuito: - Cada servicio se duerme tras 15 minutos sin peticiones. La primera petición después tarda hasta unos 90 segundos. El equipo usa un monitor de UptimeRobot que consulta los tres /actuator/health cada 5 minutos y los mantiene despiertos (comprobado el 28/10/2026). Ojo: el plan gratuito tiene un límite de horas de servicio al mes compartido por los tres. - La base gratuita tiene fecha de vencimiento. - Las tareas programadas (expiración de alertas, retención) solo corren mientras el servicio está despierto.

El servicio anterior del monolito (alerta-mujer-backend) ya no forma parte del despliegue.

### 6.2 Frontend Web en Vercel

**Paso 1:** Pon las URL de los tres servicios en src/environments/environment.prod.ts y súbelo al repositorio.

**Paso 2:** En https://vercel.com importa el repositorio Frond-end-Web. En **Root Directory** elige alerta-mujer-web.

**Paso 3:** Vercel lee vercel.json, que trae el comando de build, la carpeta de salida y la regla para que todas las rutas carguen index.html:

```
{
  "framework": "angular",
  "buildCommand": "npm run build",
  "outputDirectory": "dist/alerta-mujer-web/browser",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

**Paso 4:** Pulsa **Deploy**. Después, cada push a la rama configurada despliega solo.

**Paso 5:** Agrega el dominio de Vercel en Firebase Console > Authentication > Authorized domains, para que funcione **Continuar con Google**.

### 6.3 App Móvil

Sigue la sección 4.4 con el perfil preview, que ya apunta a los tres servicios de Render, y distribuye el APK.

### 6.4 Integración Continua (GitHub Actions)

Los tres repositorios tienen .github/workflows/ci.yml, que corre en cada push y pull request: el backend ejecuta las pruebas de los cuatro módulos con Maven, la web ejecuta las pruebas y la compilación de producción, y el móvil revisa los tipos de TypeScript y ejecuta Jest. Están en verde desde el 28/10/2026.

Además, los tres repositorios conservan un Jenkinsfile (no hay un servidor Jenkins verificado en uso):

| Repositorio | Etapas |
|---|---|
| Móvil | Dependencias, Lint, Tipos, Pruebas, APK (EAS) |
| Web | Dependencias, Build, Pruebas, Desplegar en Vercel |
| Backend | Pruebas, Imagen Docker, Desplegar en |

| Repositorio | Etapas |
|---|---|
| | Render |

El Jenkinsfile del backend prueba los tres microservicios en microservicios/ y construye sus imágenes Docker.

### 6.5 Checklist de Producción

- ☐ JWT_SECRET e INTERNAL_API_KEY nuevos, iguales en los tres servicios
- ☐ CORS_ORIGIN con la URL de Vercel
- ☐ Archivo de Firebase en Auth y Alert
- ☐ Proveedor de correo configurado en Alert
- ☐ Backblaze B2 configurado en Evidence
- ☐ Contexto de Liquibase de producción (sin datos de prueba)
- ☐ Los tres /actuator/health responden UP
- ☐ URL correctas en environment.prod.ts y en el perfil de eas.json
- ☐ Dominio de la web autorizado en Firebase
- ☐ APK nuevo compilado e instalado
- ☐ Primer administrador creado
- ☐ Zonas y recursos cargados
- ☐ Alerta de prueba con correo y push recibidos
- ☐ Backup de la base programado

## 7. Despliegue en Servidor Propio

Alternativa a Render y Vercel: los tres servicios, PostgreSQL y la web en un solo servidor con Docker y Nginx.

### 7.1 Servicios

Usa el docker-compose.yml del repositorio como base. Para producción:

- No publiques el puerto de PostgreSQL ni el del almacenamiento S3: solo deben verse dentro de la red de Docker.
- Publica los tres servicios solo en 127.0.0.1 (Nginx es quien recibe el tráfico de afuera).
- Usa el contexto de Liquibase de producción.
- Pon restart: always y límites de tamaño a los logs (max-size: 10m, max-file: 3).
- Para las evidencias, usa Backblaze B2 o un almacenamiento S3 con volumen persistente.

docker compose up -d --build
docker compose ps

### 7.2 Nginx como Proxy

Como los clientes eligen el servicio por el inicio de la ruta, se puede usar **un solo dominio** para la API y dejar que Nginx reparta. En ese caso, las tres URL de la web y de la app apuntan al mismo dominio.

```
server {
    listen 80;
    server_name api.alertamujer.com;

    # Las evidencias pueden pesar hasta 50 MB
    client_max_body_size 55M;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 120s;

    # La API interna nunca se expone
    location /internal/ {
        deny all;
    }

    # Evidence
    location /api/evidences {
        proxy_pass http://127.0.0.1:8083;
    }

    # Alert
    location ~ ^/api/(alerts|contacts|location-logs|zones|resources|saved-routes|
frequent-locations|received-alerts|zone-reports|resource-calls|public/tracking) {
        proxy_pass http://127.0.0.1:8082;
    }

    # Auth (todo lo demás)
    location / {
        proxy_pass http://127.0.0.1:8081;
    }
}
```

X-Forwarded-For es importante: el límite de intentos cuenta la IP real del cliente. Sin esa cabecera todas las usuarias compartirían el contador del proxy.

client_max_body_size también: el valor por defecto de Nginx es 1 MB y cortaría la subida de evidencias.

### 7.3 Nginx para la Web

```
sudo mkdir -p /var/www/alertamujer
sudo cp -r dist/alerta-mujer-web/browser/* /var/www/alertamujer/
sudo chown -R www-data:www-data /var/www/alertamujer
```

```
server {
    listen 80;
    server_name alertamujer.com www.alertamujer.com;

    root /var/www/alertamujer;
    index index.html;

    gzip on;
    gzip_types text/plain text/css application/javascript application/json image/svg+xml;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Cualquier ruta de Angular carga index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location = /index.html {
        add_header Cache-Control "no-cache";
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/alertamujer /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/alertamujer-api /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

### 7.4 HTTPS con Let's Encrypt

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d alertamujer.com -d www.alertamujer.com -d api.alertamujer.com
sudo certbot renew --dry-run
```

Después de activar HTTPS, actualiza las URL de la web (environment.prod.ts), de la app (eas.json), CORS_ORIGIN y la URL del enlace de seguimiento en Alert, y vuelve a compilar la web y la app.

El HTTPS es obligatorio en producción: el token, las contraseñas y la ubicación de la usuaria viajan en cada petición, y los perfiles de EAS no permiten HTTP sin cifrar.

### 7.5 Firewall

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22,80,443/tcp
sudo ufw enable
sudo ufw status
```

No abras los puertos 8081 a 8083 ni el de PostgreSQL.

---

## 8. Troubleshooting

### 8.1 Backend

**Un servicio no arranca**

1. Revisa los logs: docker compose logs <servicio> o la pestaña **Logs** en Render.
2. Si menciona el JWT o la clave de firma, JWT_SECRET está vacío o es el valor que estuvo publicado. Genera uno nuevo.
3. Si menciona Firebase, falta el archivo de cuenta de servicio o la ruta es incorrecta (Auth y Alert).
4. Si Alert o Evidence fallan al iniciar porque no hay tablas, arranca primero Auth: es el que aplica las migraciones.
5. Verifica las variables: docker compose config.
6. Reconstruye: docker compose build --no-cache.

**Error de Liquibase**

1. Si dice que cambió el checksum, alguien editó un changeset ya aplicado. Hay que deshacer ese cambio y crear un changeset nuevo.
2. Si es de permisos, revisa que Liquibase use el usuario dueño de la base.

**403 entre servicios**

INTERNAL_API_KEY no es igual en los tres servicios, o SERVICES_*_URL apunta a un lugar equivocado.

**Error de CORS en el navegador (403 en el preflight)**

CORS_ORIGIN no coincide exactamente con la URL de la web (protocolo, dominio y sin barra al final).

**401 "Token inválido" en un servicio y en otro no**

JWT_SECRET no es el mismo en los tres servicios.

**429 en el login**

1. Espera un minuto: se permiten 5 intentos por minuto por IP.
2. Si todas las usuarias reciben 429 detrás de Nginx, falta la cabecera X-Forwarded-For.

**Los contactos no reciben el correo**

1. Revisa EMAIL_PROVIDER, EMAIL_API_KEY y EMAIL_FROM en Alert. Sin proveedor, el correo solo se escribe en el log.
2. Revisa que el contacto tenga correo registrado.
3. Revisa en el log de Alert el estado del envío (en la tabla alert_contact queda failed con el motivo).

**No llegan las notificaciones push**

1. Revisa el archivo de Firebase en Alert y Auth. En el panel, **Configuración > Estado del sistema** debe decir "Configuradas".
2. El contacto debe tener la app instalada, la sesión abierta y el permiso de notificaciones.

**No se suben las evidencias**

1. Revisa las credenciales y el bucket de Backblaze B2 en Evidence.
2. Archivos de más de 50 MB responden 413.
3. En servidor propio, revisa client_max_body_size en Nginx.

### 8.2 Frontend Web

**Página en blanco o 404 al recargar una ruta**

Falta la regla try_files … /index.html (Nginx) o el rewrite de vercel.json.

**"Continuar con Google" no funciona**

1. Llena el bloque firebase de environment.prod.ts.
2. Agrega el dominio de la web en Firebase Console > Authentication > Authorized domains.
3. Vuelve a compilar y desplegar.

**El build falla**

1. Si dice ng: not found, instala sin --production: npm ci.
2. Aumenta la memoria de Node: export NODE_OPTIONS="--max-old-space-size=4096".
3. Limpia e instala: rm -rf node_modules .angular && npm ci.
4. Verifica Node 20 o superior.

**Las pantallas tardan o dan error al primer intento**

Los servicios de Render estaban dormidos. Espera hasta unos 90 segundos y recarga.

### 8.3 App Móvil

**La app no conecta o el login se queda cargando**

1. Revisa con qué perfil de eas.json se generó el APK. Si cambias las URL, hay que compilar un APK nuevo.
2. Si el APK es anterior al 28/10/2026, instala el APK v1.0.1 de GitHub Releases.
3. Si los servicios de Render estaban dormidos, la primera petición puede pasar el límite de 20 s de la app. Espera y vuelve a intentar.
4. En desarrollo, el celular y el computador deben estar en la misma Wi-Fi. Ejecuta npm start para actualizar la IP en .env.

**“No se pudo registrar la alerta en el servidor”**

El servicio Alert no respondió a tiempo o no hay red. Hoy no hay reintento automático: la usuaria debe volver a intentarlo.

**Agitar el teléfono no funciona con la pantalla apagada**

En el celular: quitar la app del ahorro de batería, activar el inicio automático y no ocultar la notificación “Protección activa”. La pantalla **Permisos y protección** de la app muestra qué falta.

**El mapa, las zonas o la asistencia salen vacíos**

En una base nueva no hay zonas ni recursos. Créalos desde el panel de administración.

**Ver el error exacto**

```
adb logcat -v time ReactNativeJS:V AndroidRuntime:E *:S
```

### 8.4 Base de Datos

**PostgreSQL no inicia**

1. Revisa los logs del contenedor o del servicio.
2. Revisa el espacio en disco: df -h.
3. Revisa el volumen: docker volume ls.

**Error de permisos en las tablas**

El usuario de la aplicación solo puede leer y escribir datos. Es normal que no pueda crear tablas ni modificar admin.audit_log. Si le faltan permisos sobre una tabla nueva, el changeset que la creó debe otorgarlos.

**Base lenta**

```
SELECT pid, now() - query_start AS duracion, state, left(query, 80)
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY duracion DESC;
```

Revisa el tamaño de las tablas que más crecen (alert.location_log, admin.audit_log). Si hace falta un índice, se agrega con un changeset de Liquibase, no a mano.

---

## 9. Mantenimiento

### 9.1 Actualizaciones

**Backend (Render):** push a la rama conectada y despliegue de Auth primero, luego Alert y Evidence. Si la actualización trae migraciones, Liquibase las aplica al arrancar Auth: haz backup antes.

**Backend (servidor propio):**

```
/usr/local/bin/backup-alertamujer.sh
git pull
docker compose up -d --build
docker compose logs -f
```

**Web (Vercel):** push a la rama conectada.

**Web (servidor propio):**

```
git pull
npm ci
npm run build
sudo rsync -a --delete dist/alerta-mujer-web/browser/ /var/www/alertamujer/
```

**App móvil:**

```
git pull
npm install
npx eas-cli build -p android --profile preview --local
```

Las usuarias deben instalar el APK nuevo. Si quieres avisarles, sube la **versión mínima de la app** en el panel de administración (Configuración): verán un aviso para descargar la nueva versión, sin que se les bloquee la app.

### 9.2 Monitoreo

curl https://alerta-mujer-auth.onrender.com/actuator/health
curl https://alerta-mujer-alert.onrender.com/actuator/health
curl https://alerta-mujer-evidence.onrender.com/actuator/health

- Un monitor externo gratuito (por ejemplo UptimeRobot) puede consultar los tres /actuator/health cada pocos minutos y avisar si fallan. En el plan gratuito de Render eso además mantiene despiertos los servicios.
- En el panel de administración, **Configuración > Estado del sistema** muestra la base, las notificaciones push, las alertas activas y la última revisión de alertas vencidas.
- En servidor propio: docker stats, htop, df -h y los logs de Nginx en /var/log/nginx/.

### 9.3 Seguridad

- Actualiza el sistema y las imágenes base con regularidad.
- Revisa dependencias con npm audit (web y móvil) y mvn dependency:tree (backend).
- **Rotar secretos:**
  - Si JWT_SECRET se filtra, cámbialo en los tres servicios y reinícialos. Todas las sesiones se cierran.
  - Si INTERNAL_API_KEY se filtra, cámbiala en los tres servicios al mismo tiempo.
  - Si el archivo de Firebase se filtra, genera una clave nueva en Firebase Console y elimina la anterior.
- Nunca subas .env, archivos de secretos ni el archivo de Firebase a Git.
- Elimina las ramas antiguas que tengan contraseñas o secretos en el historial y cambia esas contraseñas si se usan en algún lugar.

---

## 10. Análisis Multiplataforma

### 10.1 Instalación de Herramientas por Sistema

| Componente | Linux (Ubuntu/Debian) | Windows 10/11 | macOS |
|---|---|---|---|
| Docker | apt install docker-ce | winget install Docker.DockerDesktop | brew install --cask docker |
| Java 17 | apt install openjdk-17-jdk | winget install EclipseAdoptium.Temurin.17.JDK | brew install openjdk@17 |
| Maven | apt install maven | winget install Apache.Maven | brew install maven |
| Node.js 20+ | NodeSource setup_20.x | winget install OpenJS.NodeJS.LTS | brew install node@20 |
| Git | apt install git | winget install | brew install git |

| Componente | Linux (Ubuntu/Debian) | Windows 10/11 | macOS |
|---|---|---|---|
| | | Git.Git | |
| Android Studio (SDK) | Descarga oficial | winget install Google.AndroidStudio | brew install --cask android-studio |
| EAS CLI | npm install -g eas-cli | npm install -g eas-cli | npm install -g eas-cli |

**Windows:** Docker Desktop con WSL2 y la virtualización habilitada en la BIOS. Clona los repositorios dentro de WSL2 y no en C:\: Docker es más rápido y se evitan problemas de permisos.

**macOS:** funciona en Intel y Apple Silicon. Las imágenes de PostgreSQL y de Java tienen versión para amd64 y arm64.

### 10.2 Arranque Rápido en Cada Sistema

**Linux / macOS**

```
git clone <repo-backend> Backend-ALERTA-MUJER
git clone <repo-web> Frond-end-Web
cd Backend-ALERTA-MUJER && cp .env.example .env && nano .env
docker compose up -d --build
cd ../Frond-end-Web/alerta-mujer-web && npm ci && npm start
```

**Windows (PowerShell)**

```
git clone <repo-backend> Backend-ALERTA-MUJER
git clone <repo-web> Frond-end-Web
cd Backend-ALERTA-MUJER
Copy-Item .env.example .env
notepad .env
docker compose up -d --build
cd ..\Frond-end-Web\alerta-mujer-web
npm ci
npm start
```

En los tres sistemas la web queda en http://localhost:4200 y los servicios en los puertos 8081, 8082 y 8083.

### 10.3 Conectividad del Celular en Desarrollo

- El celular y el computador deben estar en la misma Wi-Fi.
- npm start en Frontend-movil escribe la IP del computador en .env. Para forzar una IP: node ./scripts/update-env.js 192.168.1.50.
- **Windows:** abre los puertos 8081 a 8083 en el firewall solo para redes privadas:

```
New-NetFirewallRule -DisplayName "AlertaMujer API dev" -Direction Inbound -Protocol TCP -LocalPort 8081-8083 -Action Allow -Profile Private
```

- **macOS:** en Configuración del Sistema > Red > Firewall, permite conexiones entrantes a Docker Desktop.

### 10.4 Errores Comunes por Sistema

**Saltos de línea (CRLF vs LF)**
Síntoma: los scripts fallan o Liquibase dice que cambió el checksum de un .sql que nadie tocó.
Solución: git config --global core.autocrlf input (Linux/macOS) o false (Windows con WSL2), y "files.eol": "\n" en VS Code.

**Puerto ocupado**
Síntoma: “Address already in use” o “port is already allocated”.
Solución: busca el proceso (sudo lsof -i :8081 en Linux/macOS; netstat -ano | findstr :8081 y taskkill /PID <pid> /F en Windows) o cambia el puerto publicado en el compose. Recuerda que Metro (Expo) también usa el 8081.

**Docker Desktop no inicia en Windows**
Solución: wsl --install, habilitar VT-x/AMD-V en la BIOS y reiniciar.

**El celular no llega al backend local**
Solución: misma Wi-Fi, npm start para actualizar la IP y abrir los puertos en el firewall del computador.

**Rutas de volúmenes**
Síntoma: un servicio no encuentra el archivo de Firebase.
Solución: ejecuta docker compose siempre desde la raíz de Backend-ALERTA-MUJER; las rutas del compose son relativas a esa carpeta.

---

## 11. Apéndice

### 11.1 Comandos Útiles

**Backend**

```
docker compose up --build     # levantar en local
docker compose logs -f        # logs
docker compose down           # detener
cd microservicios && mvn clean test          # pruebas
```

**Web**

```
npm start                  # desarrollo
npm run build              # compilar
npx ng test --watch=false      # pruebas
```

**Móvil**

```
npm start                                    # desarrollo
npm test                                     # pruebas
npx tsc --noEmit                             # tipos
npx eas-cli build -p android --profile preview --local     # APK en local
eas build -p android --profile preview                     # APK en la nube
adb logcat -v time ReactNativeJS:V AndroidRuntime:E *:S    # logs del celular
```

**Base de datos**

```
psql "<cadena-de-conexión>"
pg_dump "<cadena-de-conexión>" > backup.sql
psql "<cadena-de-conexión>" < backup.sql
```

### 11.2 Contacto de Soporte

**Equipo de desarrollo AlertaMujer:** - Maicol Stiven Sánchez Castañeda - Justin Daniela Bahamón Baloco - José Sebastián Suárez Cumaco

Las incidencias técnicas se reportan como *issue* en el repositorio correspondiente (backend, web o móvil), con los pasos para reproducir el error y los logs.

### 11.3 Recursos Adicionales

- Documentación de Docker
- Documentación de Nginx
- Documentación de PostgreSQL
- Documentación de Liquibase
- Documentación de Render
- Documentación de Vercel
- Documentación de EAS Build
- Documentación de Let’s Encrypt

---

**Fin del Manual de Instalación**

*Este manual está dirigido a quienes instalan y despliegan AlertaMujer.*